Course: Individual Software Development Process' 2026

Software Requirement Specification

By **AXIS**

Chosen Irl Challenge: Project C: Department Information & Communication Hub

Jehan Tohdeng	6810545514
Thanakorn Innok	6810545620
Piyatida Muanjaingam	6810545760
Ratamon Charoenratanakun	6810545883

Table of Contents

1. Target Users

- **Students (CPE and SKE):** The main users of the hub. They sign in with their university Google account, read department announcements, browse published documents and links, and look for the information that applies to their department and year of study. Lecturers and department staff who are not granted publishing rights use the hub in exactly this way.
- **Content editors:** Lecturers, department staff, and student representatives authorised by the administrator to publish. They create and edit content items, choose the audience and visibility, pin announcements, and expire or extend them. This role exists because the current process depends on staff and student representatives forwarding announcements by hand.
- **Department administrator:** The staff member accountable for the content of the hub and for the role held by each account. Holds every permission a content editor holds, and in addition hides items published in error and assigns roles.

2. The System Goal

G-0: Maintain a single, accurate, and accessible hub for CPE and SKE department information.

The two departments currently distribute information through fragmented channels. A survey of the channels actually in use between 29 June and 20 August 2026, found department-level announcements spread across three separate Discord servers, the university portal, course-level Google Classroom streams, and forwarded email, with student representatives forwarding announcements by hand and asking staff case by case which year group each announcement should reach.

Course platforms are scoped to a single subject and reach only the students enrolled in it, so department-wide information such as scholarships, internships, laboratory announcements, forms, and cross-year notices either gets cross-posted into unrelated course streams or has no home at all. As a result students miss announcements, receive conflicting versions of the same information, and cannot find a document again once the chat has scrolled past it. The hub gives department-level information one authoritative source, separate from the per-course platforms that remain in use for coursework.

In scope for this iteration.

Sign-in with a university Google account restricted to the permitted domain; a role model separating readers, editors, and administrators; creating, editing, and publishing content items with attachments and links; classifying items by department and target audience; choosing per item whether it is visible to everyone or only to signed-in university accounts; pinning; expiry that moves an item out of the current feed while keeping it searchable; permanent hiding for items published in error; browsing, filtering, keyword search, and download; and an audit record of every write; signing out; and a feed ordered towards each member's department and year of study

Out of scope for this iteration.

Two-way messaging, chat rooms, and comment threads, because the department already runs Discord for that purpose and any system that accepts free text from students would require moderation staff that the department has not committed. Also excluded: a submission-and-approval workflow for lecturers, timetable integration, and push or email notification. These are excluded so that the five sub-goals below can be delivered and verified within the iteration rather than partially delivered across a wider surface.

Planned for later iterations of this project.

A monthly calendar view, follow-up entries that let an announcement carry later updates and a set of photographs taken after the event, a button allowing students to register interest in an event, an iCalendar subscription feed, and a bilingual interface. These carry no external dependency and are expected to be delivered later in the term. They are named here rather than specified below, because specifying them now would place requirements in this document that the diagrams in Sections 4 and 7 do not yet cover.

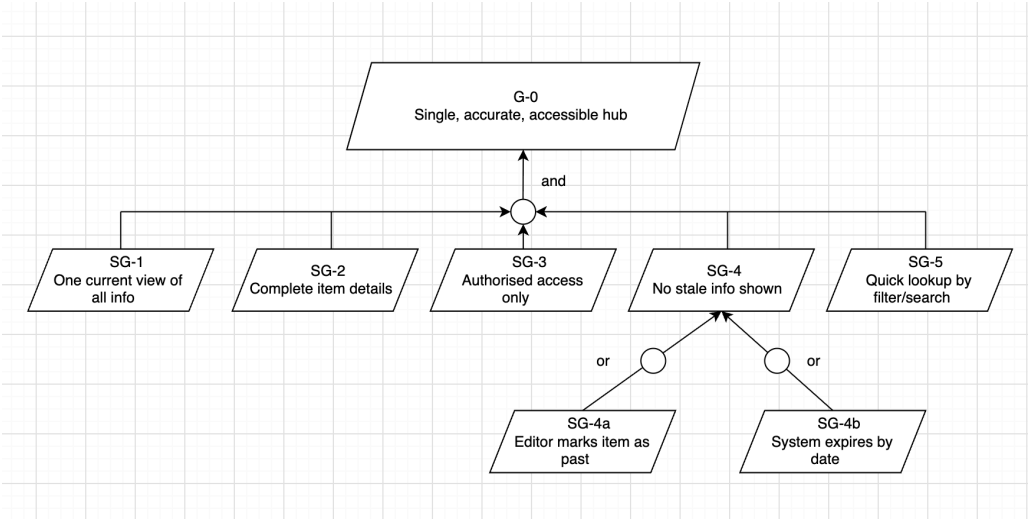
Assumptions and constraints.

This specification rests on four assumptions confirmed with the stakeholder, each of which constrains the requirements below. First, staff and student accounts share one email domain, so the domain restriction in SRS-10 admits every member of the department; were that not so, the restriction would exclude staff and the correction would lie in the identity provider rather than in this system. Second, an item’s target audience is the combination of the department and the year or years of study that the item addresses, and this division is sufficient, even though several announcements in current use address programmes and admission schemes instead; an item tagged for the wrong audience therefore reaches the wrong students without any signal, which is why SRS-13 confines tagging to authorised editors and why SRS-14 and SRS-15 together refuse to save an item whose target audience is missing. An item whose

activity is open to participants beyond the two departments is published with visibility set to everyone; an item addressed only to members of the two departments takes the default. Third, students open the hub themselves rather than waiting to be notified, so pinning under URS-9 is the only mechanism for raising a short-notice announcement in this iteration. Fourth, expiry depends on an editor setting an expiry date at publication time; an item saved without one remains in the current feed until an editor marks it as past, which is why SG-4 is or-refined into an editor action and a system action rather than relying on either alone.

Constraints on delivery. The hub is delivered as a web application with a relational database, developed on GitHub and packaged with Docker so that the same build runs on every team member's machine and on the deployment host. Authentication depends on Google OAuth 2.0, so the hub cannot be used while that service is unavailable; no local password fallback is provided in this iteration.

3. Goal Refinement via KAOS



G-0 is and-refined into the five sub-goals below: all five must hold for the goal to be satisfied. SG-4 is refined into SG-4a and SG-4b, either of which alone satisfies it.

- **Sub-Goal 1 (SG-1)**

Description: Achieve: Provide students with one current view of all department information.

Rationale: Students should not have to check several chat servers, a course platform, and a notice board to learn what the department has announced.

- **Sub-Goal 2 (SG-2)**

Description: Maintain: Complete and correctly classified content details.

Rationale: Every item needs a title, type, department, target audience, publication date, body, and an optional attachment or link, so that it can be displayed, filtered, and trusted.

- **Sub-Goal 3 (SG-3)**

Description: Achieve: Restrict the hub to verified university members and confine publishing to authorised roles.

Rationale: The hub is an official source for two departments, so only accounts in the permitted domain may read restricted items, and within those accounts only editors and administrators may decide what appears.

- **Sub-Goal 4 (SG-4)**

Description: Avoid: Expired or superseded information being presented as current.

Rationale: An announcement whose deadline has passed, or a document that has been replaced, causes students to act on the wrong information.

- **Sub-Goal 4a (SG-4a)**

Description: Achieve: An editor marks an item as no longer current.

Rationale: Some items have no fixed deadline, so a person has to decide when they stop applying.

- **Sub-Goal 4b (SG-4b)**

Description: Achieve: The system stops presenting an item as current once its expiry date has passed.

Rationale: Items with a known deadline should not depend on someone remembering to remove them.

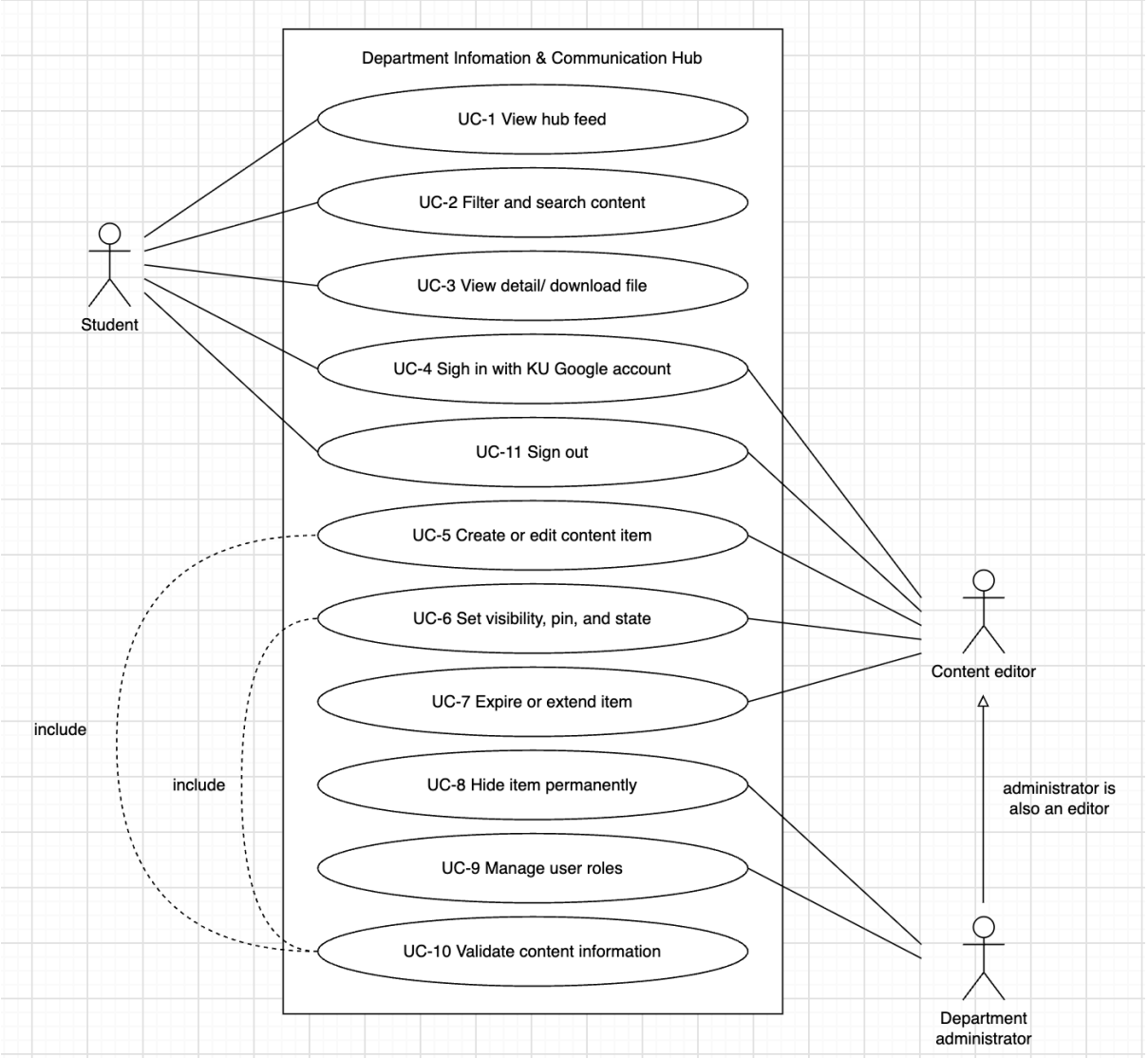
- **Sub-Goal 5 (SG-5)**

Description: Achieve: Enable students to locate relevant information quickly.

Rationale: A hub is only useful if a student can filter by department and audience, or search by keyword, and reach the item in a few steps.

4. Use Case Diagram

Every use case in the diagram carries the identifier listed below, so that each row of the traceability matrix in Section 13 refers to a use case that can be located in the diagram.



5. User Stories

User Story ID	Description	Acceptance Criteria
US-1	As a student, I want to see all currently published department information in one feed so that I do not have to check several chat servers.	The feed shows the title, type, department, target audience, and publication date of each item, with pinned items first. Items that have passed their expiry date and items hidden by an administrator are not shown in the current feed.
US-2	As a student, I want to filter and search the hub so that I can find the information that applies to my department and year.	The student can filter by department, type, and year of study, and search by keyword. A message is shown when no item matches. Past items can be included in the results on request.
US-3	As a student, I want to open an item and download its attachment so that I can read the full details and keep the document.	The detail view shows the body and any attachment or link. A restricted item and its attachments cannot be reached without a session, including by opening the address directly.
US-4	As a member of the department, I want to sign in with my university Google account so that I can reach restricted information without creating another password.	Sign-in uses Google OAuth 2.0. An account outside the permitted domain is refused with a clear reason and no session is created. A signed-in member can end the session at any time and is returned to the public view.
US-5	As an editor, I want to create and edit a content item so that students receive new department information.	Required fields must be provided before the item can be saved. A valid item becomes visible to its audience once its state is set to published.
US-6	As an editor, I want to choose whether an item is visible to everyone or only to signed-in university accounts, so that internal information does not reach the public.	Visibility must be chosen before saving and defaults to university accounts only. A public item appears to visitors who are not signed in.
US-7	As an editor, I want an item to stop appearing as current once its deadline has passed, while remaining findable, so that students neither act on stale information nor lose the record of it.	A past item leaves the current feed, either on its expiry date or when an editor marks it as past. It remains available through search and the past view, and is labelled as concluded

		wherever it appears.
US-8	As an administrator, I want to assign a role to each account so that only the right people can publish.	Every signed-in account holds exactly one role and a new account defaults to reader. Only an administrator can change a role.
US-9	As an editor, I want to pin an announcement to the top of the feed so that a notice given at short notice is seen before the rest.	An editor can pin and unpin an item. Pinned items appear before all others in the feed, and pinning does not change who is able to see the item.
US-10	As a signed-in member, I want to sign out of the hub so that I can securely end my session.	A signed-in member can sign out at any time. The current session is ended and the member is returned to the public view.
US-11	As an administrator, I want to hide permanently a content item that was published in error, so that students can no longer reach it while the record is kept for accountability.	Only an administrator can hide an item, and the system asks for confirmation first. Once hidden, the item does not appear in the feed, in search results, or in the past view, and its address returns nothing to a student. The item remains visible in the administrator view, and the action is recorded.
US-12	As an administrator, I want every change to a content item or to a role to be recorded, so that I can establish who published or altered an announcement.	Creating, editing, changing state, pinning, expiring, extending, or hiding an item writes a record, as does changing an account's role. Each record holds the acting account, the item or account affected, the action, and the time. Records cannot be edited or deleted from the interface.
US-13	As a signed-in student, I want the feed to put the announcements meant for my department and year first, so that I do not have to filter the hub every time I open it.	For a signed-in member whose department and year of study are known, items addressed to that department and year appear before items addressed to others, after any pinned items. The member can turn the ordering off and see the feed in publication order. No item is hidden by the ordering.

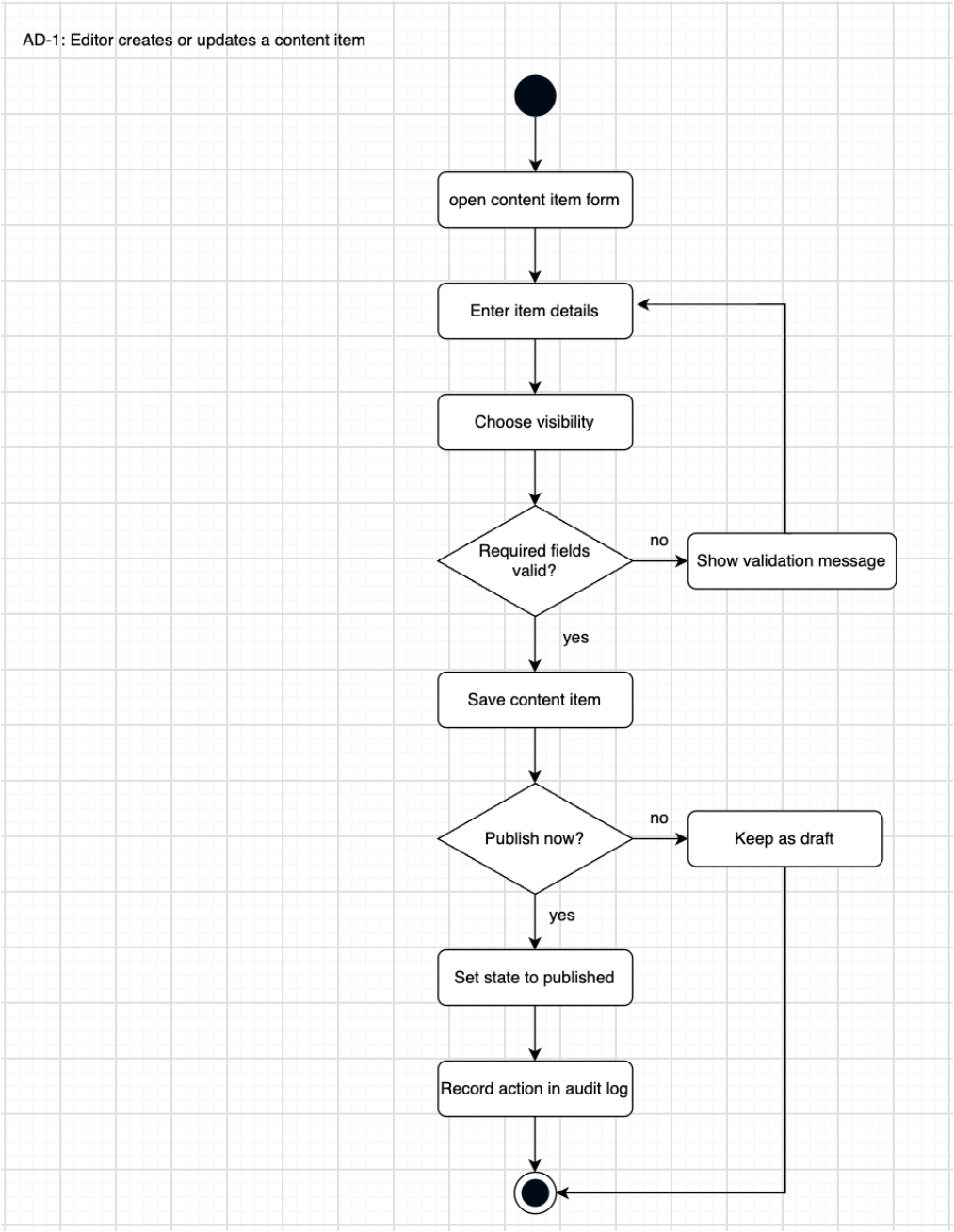
6. User Requirement Specification

ID	User Requirement
URS-1	Students shall be able to view a list of the department information that is currently published.
URS-2	Students shall be able to view each item's essential information, including title, type, department, target audience, and publication date.
URS-3	Students shall be able to filter content items by department, by type, and by year of study, and to search them by keyword.
URS-4	Students shall be able to open a content item and download its attachment, or follow its link, where supplied.
URS-5	Members of the department shall be able to sign in with a university Google account, and the system shall accept only accounts in the permitted domain.
URS-6	The system shall hold one role for each account and shall allow an administrator to change it.
URS-7	An authorised editor shall be able to create and edit content items.
URS-8	An authorised editor shall be able to set the visibility of a content item to everyone or to university accounts only.
URS-9	An authorised editor shall be able to pin an important announcement to the top of the feed.
URS-10	The system shall move an item out of the current feed once it is no longer current, while keeping it findable through search and a past view.
URS-11	An administrator shall be able to hide permanently a content item that was published in error.
URS-12	Members shall be able to end their session and return to the public view.
URS-13	The system shall keep a record of every change to a content item and of every change to an account's role.
URS-14	The system shall present a signed-in member with a feed ordered towards the information that applies to that member's department and year of study.

7. Activity Diagrams

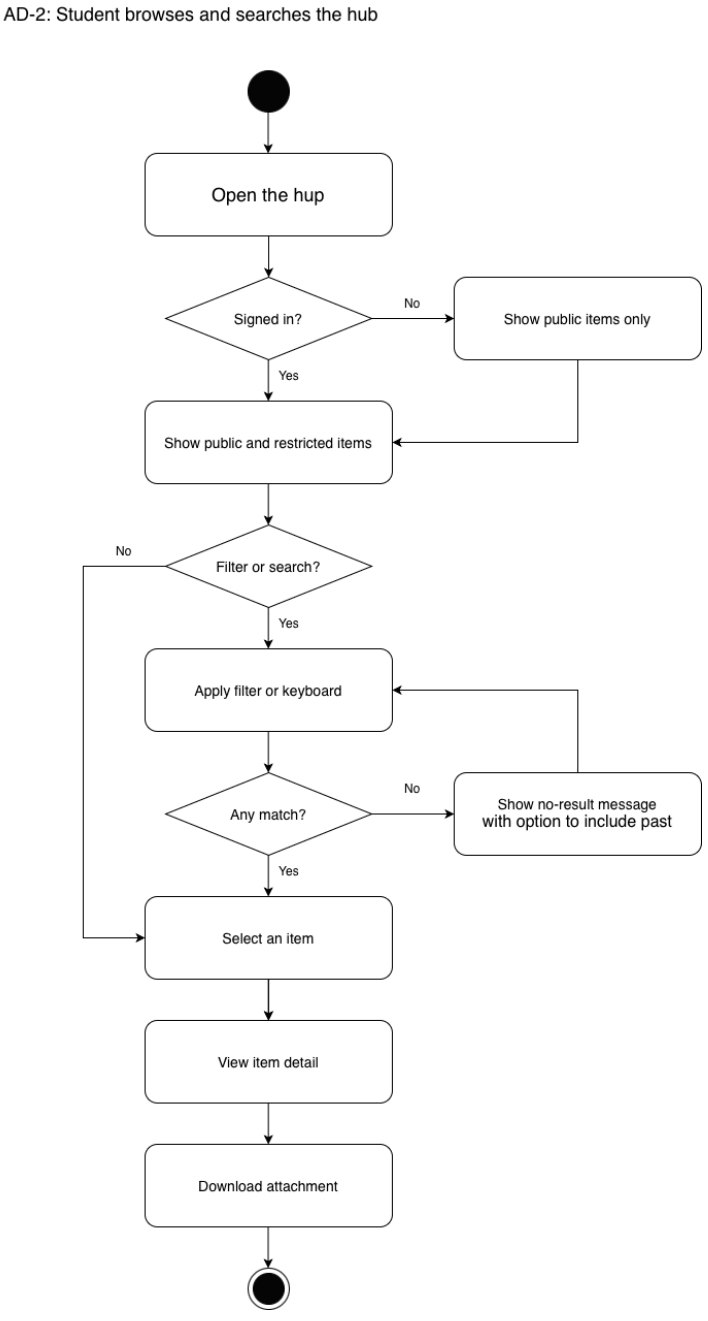
AD-1: Editor creates or updates a content item

This diagram was used during elicitation to discover the required fields, the validation decisions, and the exception paths, which were converted into SRS-13 to SRS-17. It also records the audit entry written on every publication, from which SRS-21 is derived.



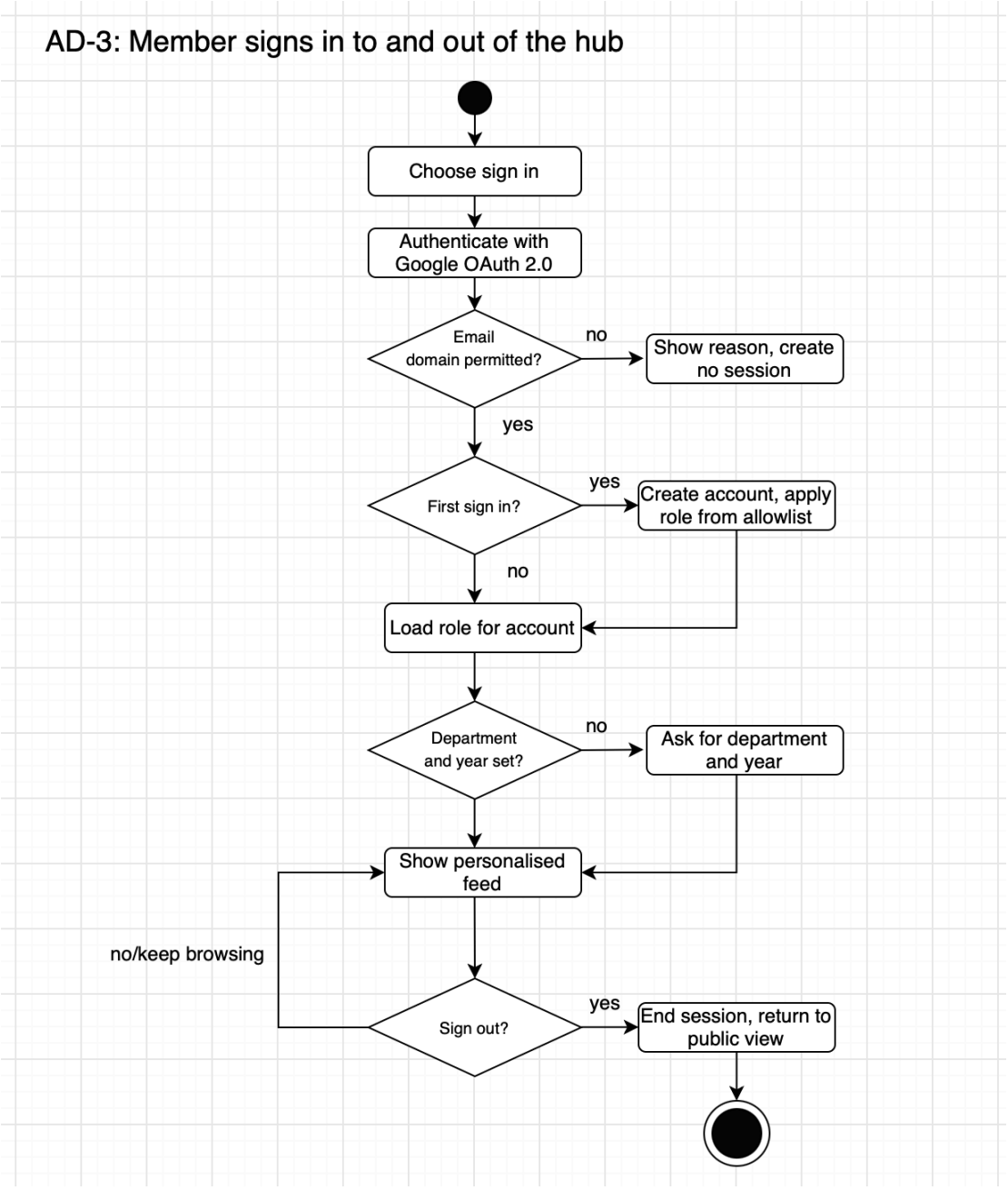
AD-2: Student browses and searches the hub

This diagram traces the student-facing browsing, filtering, and search flow, including the decision that separates the current feed from the past view, and was used to derive SRS-1 to SRS-8.



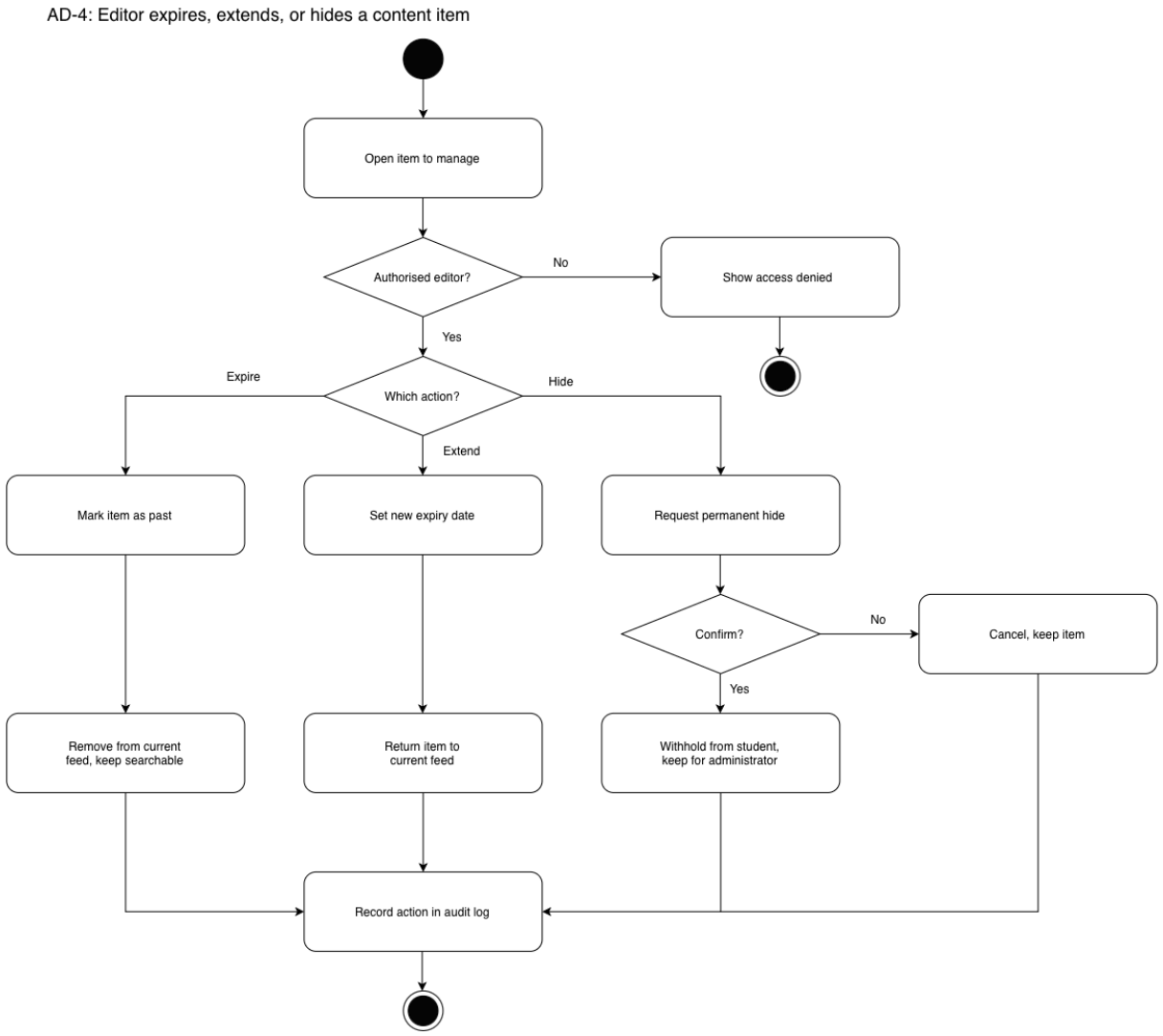
AD-3: Member signs in to and out of the hub

This diagram captures the identity check, the domain restriction, and the role lookup that follows a successful sign-in, and was used to derive SRS-9 to SRS-12. It closes with the path by which a member ends the session, from which SRS-22 is derived.



AD-4: Editor expires, extends, or hides a content item

This diagram captures the difference between an item that becomes past and an item that is hidden permanently, together with the confirmation the second action requires, and was used to derive SRS-18 to SRS-20. Every branch writes an audit entry, so this diagram also supports SRS-21.



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall display in the current feed only those content items whose state is published, whose expiry date has not passed, and which have not been hidden.
SRS-2	The system shall display the title, type, department, target audience, and publication date of each item in the feed, and shall list pinned items before the others.
SRS-3	The system shall allow students to filter content items by department, by type, and by year of study.
SRS-4	The system shall allow students to search content items by keyword over the title and the body, and shall display a message when no item matches.
SRS-5	The system shall include past items in search results when the student requests them, and shall label each such item as concluded.
SRS-6	The system shall display, in the detail view of an item, its body together with any attachment or link supplied.
SRS-7	The system shall verify that the requester is permitted to view an item before issuing any link to its attachment, and shall issue links that expire five minutes after they are issued.
SRS-8	The system shall refuse access to a restricted item and to its attachments when the request carries no valid session, however the address was obtained.
SRS-9	The system shall authenticate members through Google OAuth 2.0 and shall create a session only after the provider returns a verified account.
SRS-10	The system shall refuse any authenticated account whose email domain is not among the permitted domains, shall display the reason, and shall not create a session.
SRS-11	The system shall store exactly one role for each account and shall assign the reader role to a newly created account unless the address appears in the list of addresses registered in advance.
SRS-12	The system shall allow an administrator to change the role of any account, and shall apply the new role from that account’s next request onward.

SRS-13	The system shall permit creating, editing, changing the state of, pinning, and hiding a content item only to accounts holding the editor or administrator role.
SRS-14	The system shall require a title, a body, a type, a department, a target audience, and a visibility before saving a new or edited content item.
SRS-15	The system shall display a validation message identifying the missing or invalid data, and shall not save the item.
SRS-16	The system shall store a newly saved content item in either the draft or the published state, and shall thereafter move a published item to the past state under SRS-18 or SRS-24, or to the hidden state under SRS-20.
SRS-17	The system shall require a visibility of either everyone or university accounts only, and shall apply university accounts only when none is chosen.
SRS-18	The system shall treat a published item as past once its expiry date has passed, shall remove it from the current feed, and shall keep it available through search and the past view.
SRS-19	The system shall allow an authorised editor to extend the expiry date of a past item, after which the item shall appear in the current feed again.
SRS-20	The system shall require confirmation before hiding a content item permanently, and shall thereafter withhold it from students in the feed, in search results, and in the past view, while retaining it in the administrator view.
SRS-21	The system shall record the acting account, the item, the action, and the time of every operation that creates, changes, or hides a content item or changes a role.
SRS-22	The system shall allow a signed-in member to sign out, ending the session.
SRS-23	The system shall allow an authorised editor to pin and unpin a content item.
SRS-24	The system shall allow an authorised editor to mark a published item as past before its expiry date, after which the treatment defined in SRS-18 applies.
SRS-25	The system shall order the current feed of a signed-in member so that, after any pinned items, items addressed to that member’s department and year of study appear before items addressed to others, and shall

	allow the member to turn this ordering off.
SRS-26	The system shall keep a published item that has no expiry date in the current feed until an editor marks it as past under SRS-24, so that reference material such as laboratory and staff listings remains available indefinitely.

9. Software Architecture

The recommended architecture is a modular monolith following the Model-View-Controller pattern, with authentication and authorisation as a cross-cutting module and file storage as a separate service. It is one deployable application, internally divided into layers, rather than a flat monolith or a set of microservices.

Rationale

The system is built by a team of four students over a single term, and this iteration covers eleven use cases that all operate on one entity, the content item. Splitting these into independently deployed services would add service discovery, inter-service calls, and several deployment targets, none of which the team can operate and none of which any requirement asks for, since nothing in the specification calls for parts of the system to scale separately. A flat monolith carries the opposite risk: the rules that decide what a request may see are the substance of this system rather than an afterthought, because SRS-1 filters by state and expiry, SRS-8 refuses restricted items requested directly, SRS-13 confines writes to two roles, and SRS-17 defaults visibility when the editor makes no choice. If those rules were written wherever they happened to be needed, a page added later would omit one of them and the omission would be invisible until something leaked.

The modular arrangement answers that directly. Every read passes through one Model function that applies state, expiry, and visibility together, so a new page cannot bypass the rules by accident, and every write passes through the Controller, which consults the authorisation module before the Model is reached. The separation also leaves a later iteration free to extract a module, for example attachment storage, into its own service without disturbing what is already working.

Component	Layer/ Type	Description
Web Client	Presentation (View)	Renders the public feed, the signed-in feed, item detail, search results, the editor forms, and the administrator views. Displays validation messages, the reason a sign-in was refused, and the label marking a concluded item.
Content Controller	Controller	Receives requests to view, filter, search, create, edit, pin, expire, extend, and hide items. Consults the authorisation module before any write (SRS-13) and returns the response to the View.
Content Model	Model	Holds the rules of the system: selects items by state, expiry, and visibility (SRS-1, SRS-5, SRS-18), validates required fields before saving (SRS-14, SRS-15), applies the visibility default and the pin order (SRS-2, SRS-17, SRS-23), and writes the audit entry for every change (SRS-21).
Authentication and authorisation module	Cross-cutting	Performs the OAuth exchange, checks the email domain, resolves the role held by the account, and ends sessions (SRS-9 to SRS-13, SRS-22). Used by the Controller on every request rather than by each page.
Relational database	Persistence	Stores accounts and roles, content items with their audience and visibility, types, and audit entries.
File store	Persistence	Holds attachments outside the database and issues short-lived links only after the Controller has confirmed the requester may see the item (SRS-7).

10. Data Storage Technology

The recommended primary data storage technology for AXIS is a Relational Database Management System (RDBMS), and this project will use PostgreSQL. Attachments are held separately in an S3-compatible object store, while the database stores the corresponding file name and storage reference.

Rationale

The information managed by AXIS is structured and has clearly defined attributes. Each content item contains information such as title, type, department, target audience, publication date, body, state, expiry date, pin state, hidden state, and an optional attachment or link.

PostgreSQL is selected because the team already runs it in the development environment described in Section 11, because its full-text search covers the keyword requirement in SRS-4 without a separate search engine, and because its array and JSON column types hold an item's target audience without a further table.

Uploaded files are not stored inside the database. They are held in an S3-compatible object store, with the database keeping the file name and storage reference. An object store is required rather than a plain container volume because SRS-7 obliges the system to issue links that expire five minutes after they are issued: a presigned URL provides exactly that, whereas a file served straight from a volume would have to be protected by application code on every request. MinIO is used in development because it speaks the same API as the hosted object stores the department might later move to.

Component	**Data Storage Technology**	**Description**
Content Database	PostgreSQL	Stores structured content records and supports the filtering, state, expiry, hiding, and validation requirements defined by this specification.
Attachment Storage	S3-compatible object store (MinIO)	Stores files attached to content items and issues presigned links that expire five minutes after they are issued (SRS-7). The database holds the storage reference so the file can be reached from the detail view.

11. Development Environment

The recommended development and deployment environment for AXIS is a containerised environment using Docker and Docker Compose rather than relying on one specific bare-metal machine configuration.

Rationale

AXIS consists of a web application, a relational database, an object store for attachments, and expiry applied at query time. A containerised environment allows these components to run with consistent dependencies and configurations across different development machines.

Docker reduces environment differences between team members and makes the application easier to reproduce, test, and deploy. Docker Compose can start the application server and relational database together, while persistent volumes preserve database records and uploaded attachments.

The same container configuration can later be reused on a server or hosting environment, reducing deployment differences between development and production.

Component	**Development Environment**	**Description**
AXIS Application Server	Docker Container	Runs the MVC application: content management, authorisation checks, filtering and search.
Content Database	PostgreSQL in a Docker container, with a named volume	Holds accounts, roles, content items, and audit entries. The volume preserves records across container restarts and between team members.
Attachment Storage	MinIO in a Docker container, with a named volume	Provides the S3-compatible object store described in Section 10, so presigned links behave in development exactly as they will in deployment.

Orchestration	Docker Compose	Starts the application server, database, and object store together with one command, so every machine runs the same versions and configuration.
Version control and integration	GitHub, with GitHub Actions	Holds the repository and runs the build and the automated checks on every push, so a change that breaks the build is caught before it reaches the shared branch.

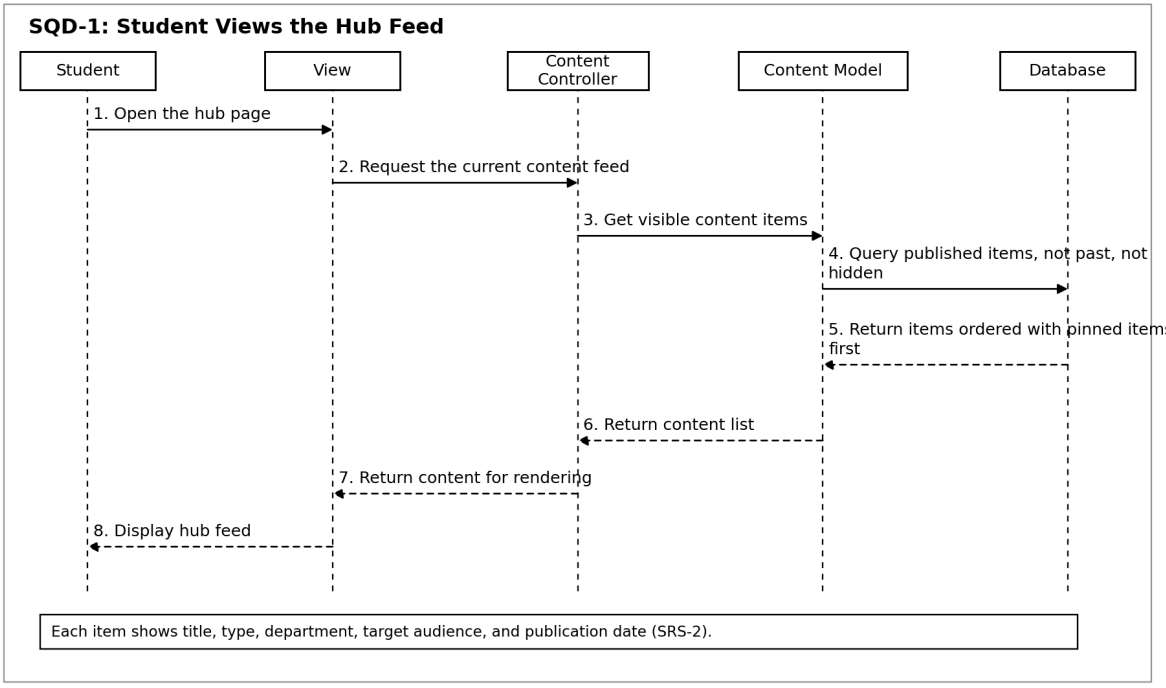
12. Sequence Diagrams

The following sequence diagrams are derived from the modular MVC architecture selected in Section 9. The requirements traced by each diagram are listed with it, and the complete mapping appears in Section 13.

SQD-1: Student Views Hub Feed

Traced requirements: SRS-1, SRS-2

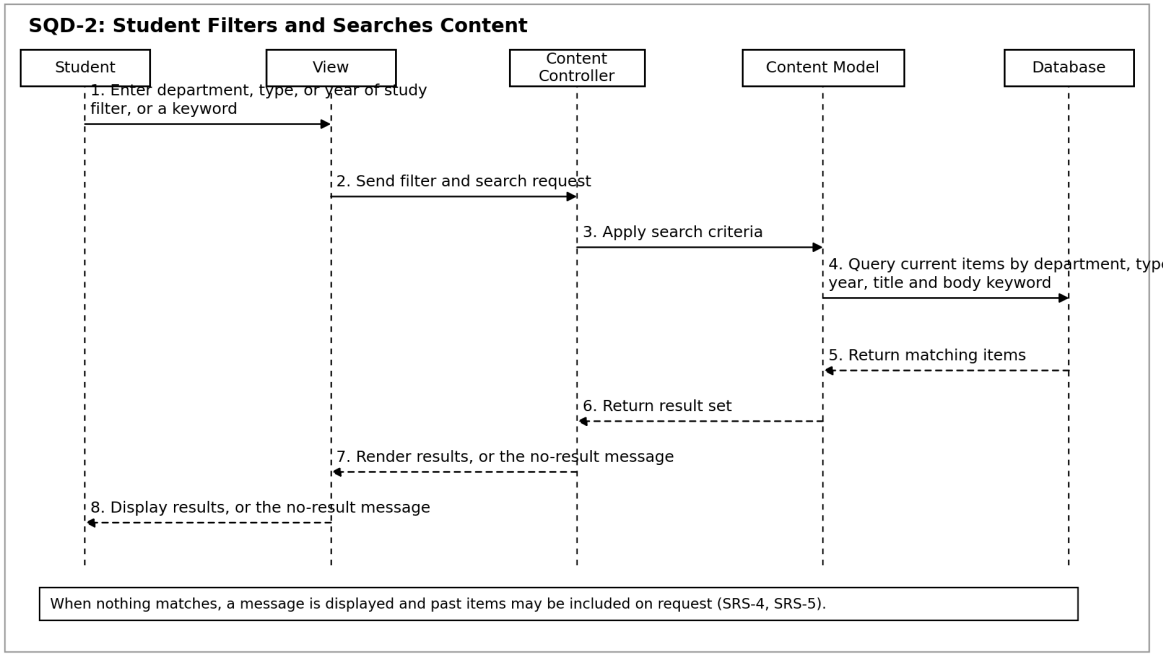
Shows how a student opens the hub and receives only current published content. Past and hidden items are excluded, and pinned items are returned first.



SQD-2: Student Filters and Searches Content

Traced requirements: SRS-3, SRS-4

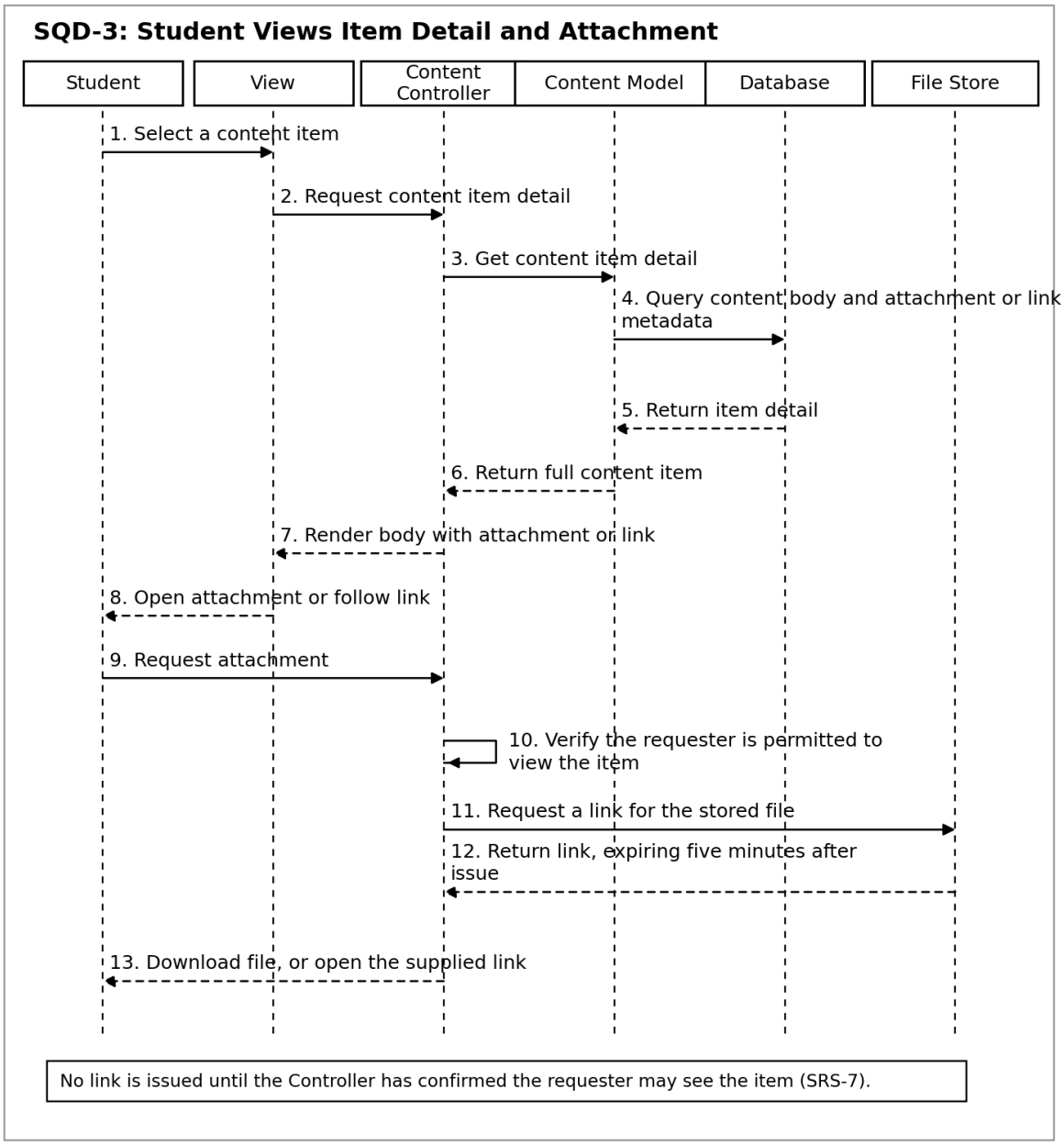
Shows how department, type, year of study, and keyword criteria are sent through the application and how matching results or a no-result message are returned.



SQD-3: Student Views Content Detail and Attachment

Traced requirements: SRS-6, SRS-7

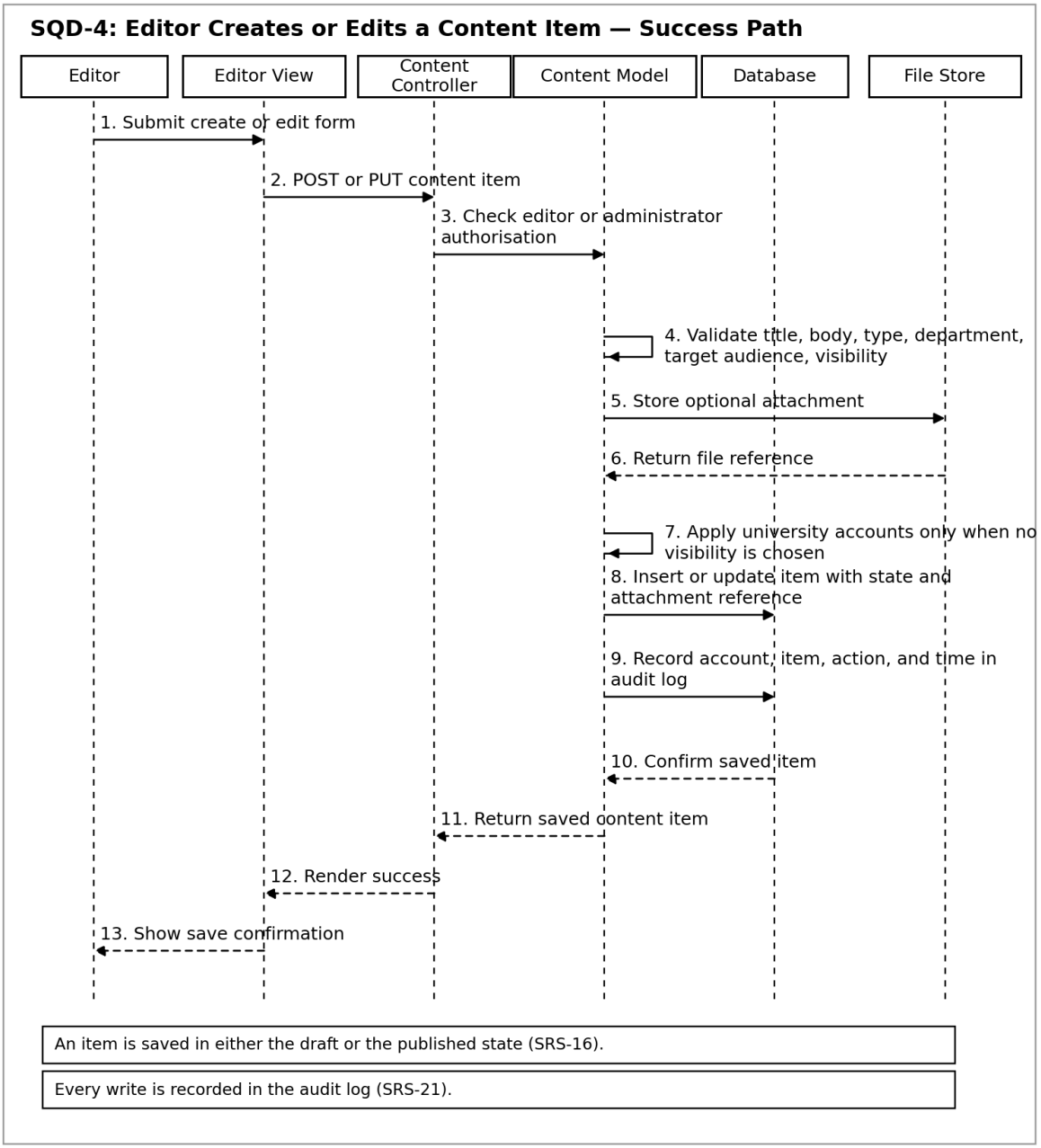
Shows how a student opens a full content item and then downloads an attachment or follows the supplied link. The Controller confirms that the requester may see the item before a link is issued, and the link expires five minutes later.



SQD-4: Editor Creates or Edits a Content Item (Success)

Traced requirements: SRS-13, SRS-14, SRS-16, SRS-17, SRS-21

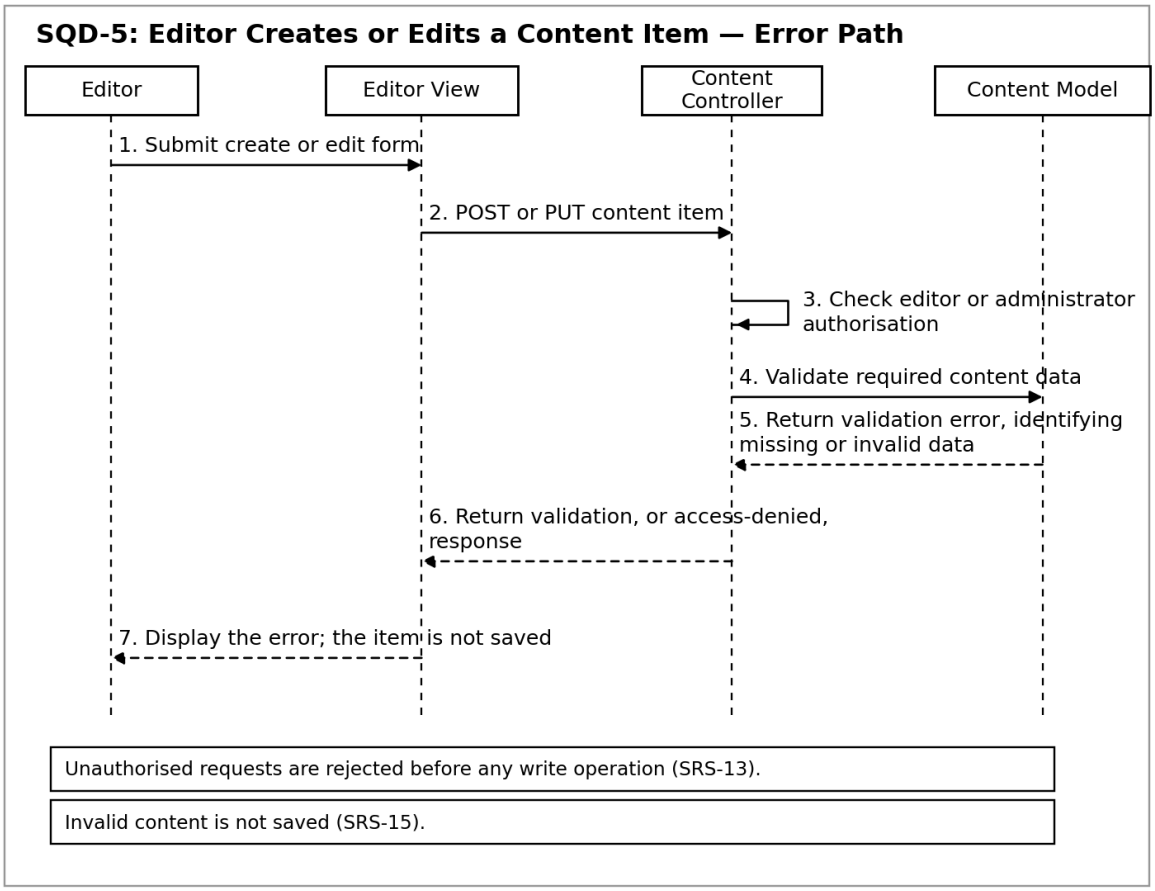
Shows the authorised success path for validating and saving a content item, including an optional attachment or link, the visibility applied when the editor makes no choice, the state in which the item is saved, and the audit entry written on completion.



SQD-5: Editor Creates or Edits a Content Item (Error)

Traced requirements: SRS-13, SRS-15

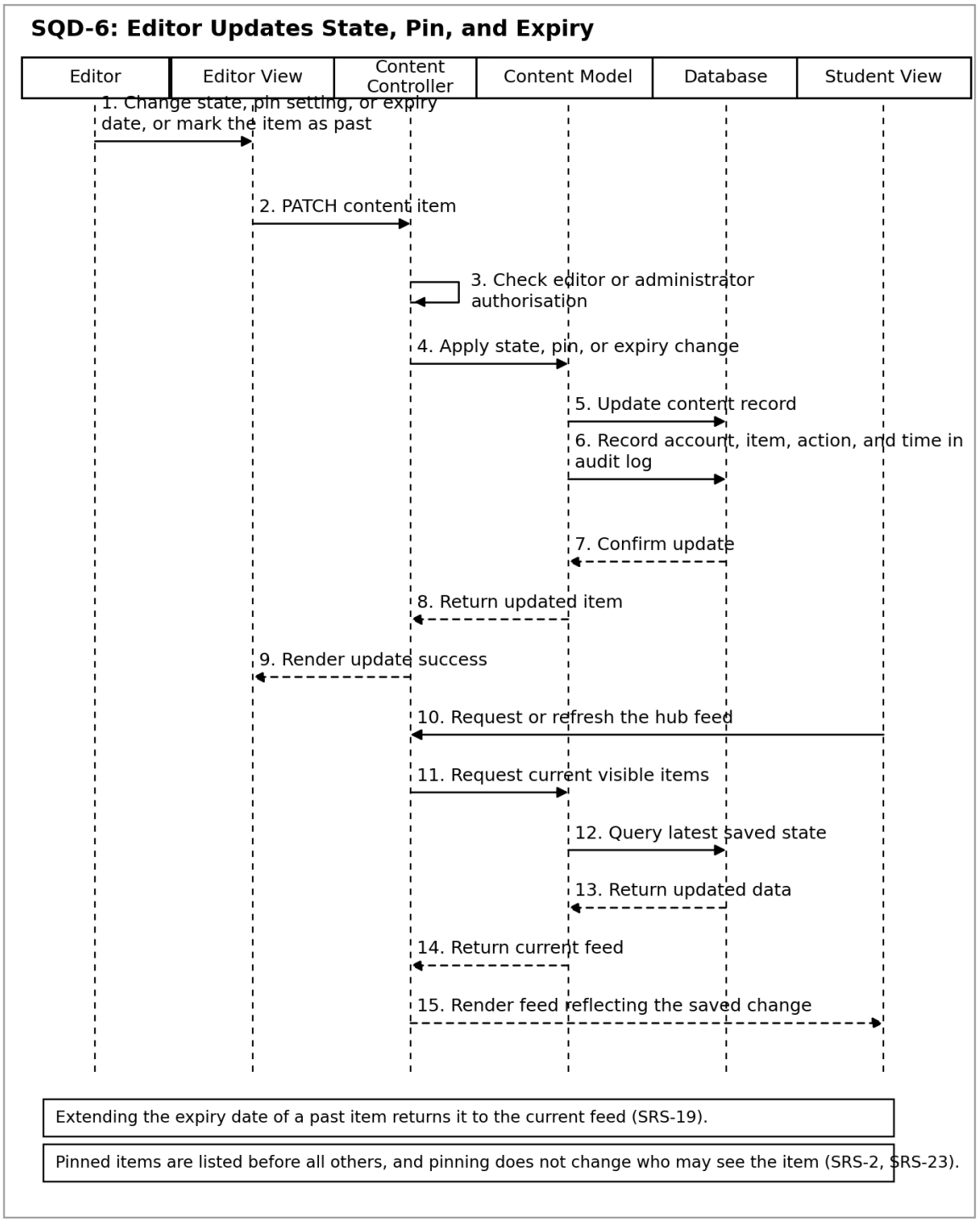
Shows how unauthorised requests are rejected before any write, and how invalid content returns a validation message identifying the missing data without saving the item.



SQD-6: Editor Updates State, Pin, and Expiry

Traced requirements: SRS-13, SRS-16, SRS-19, SRS-23, SRS-24

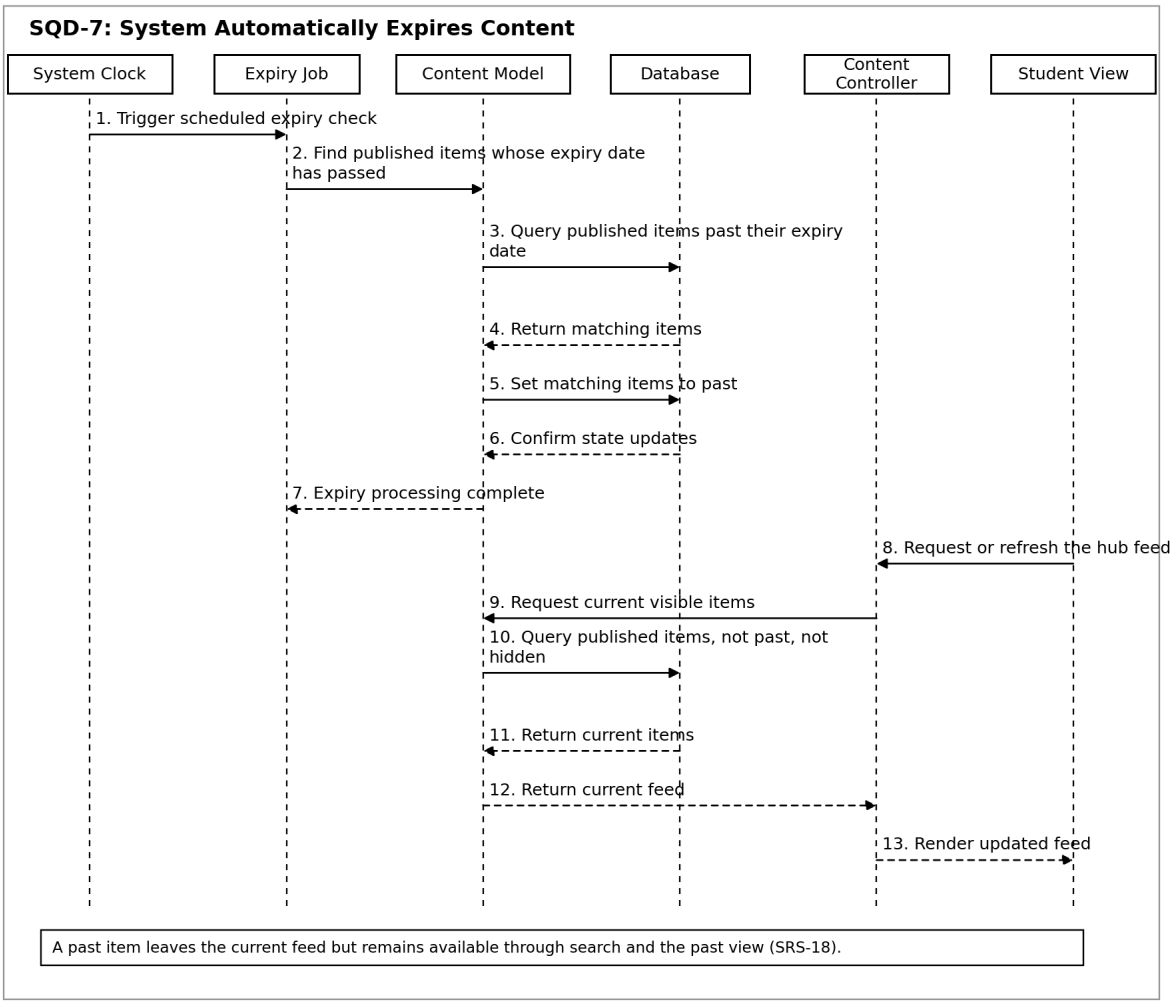
Shows how an authorised editor changes the state of an item, pins or unpins it, extends its expiry date, or marks it as past before that date, and how the refreshed student feed reflects the saved change.



SQD-7: Expiry Applied When the Feed Is Requested

Traced requirements: SRS-1, SRS-18

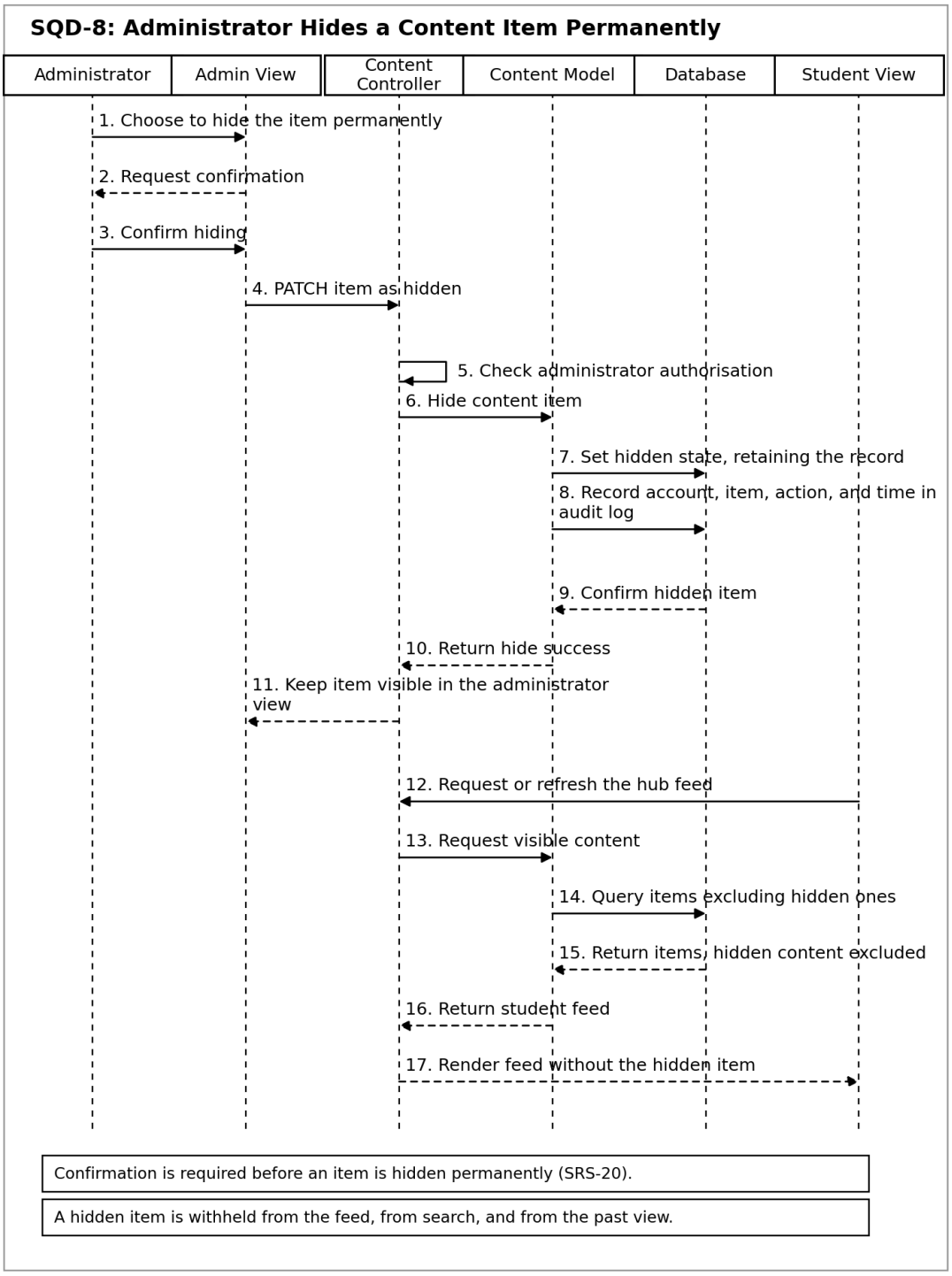
Shows how the Model excludes items whose expiry date has passed each time the feed is built, so no scheduled process is required.



SQD-8: Administrator Hides a Content Item Permanently

Traced requirements: SRS-13, SRS-20, SRS-21

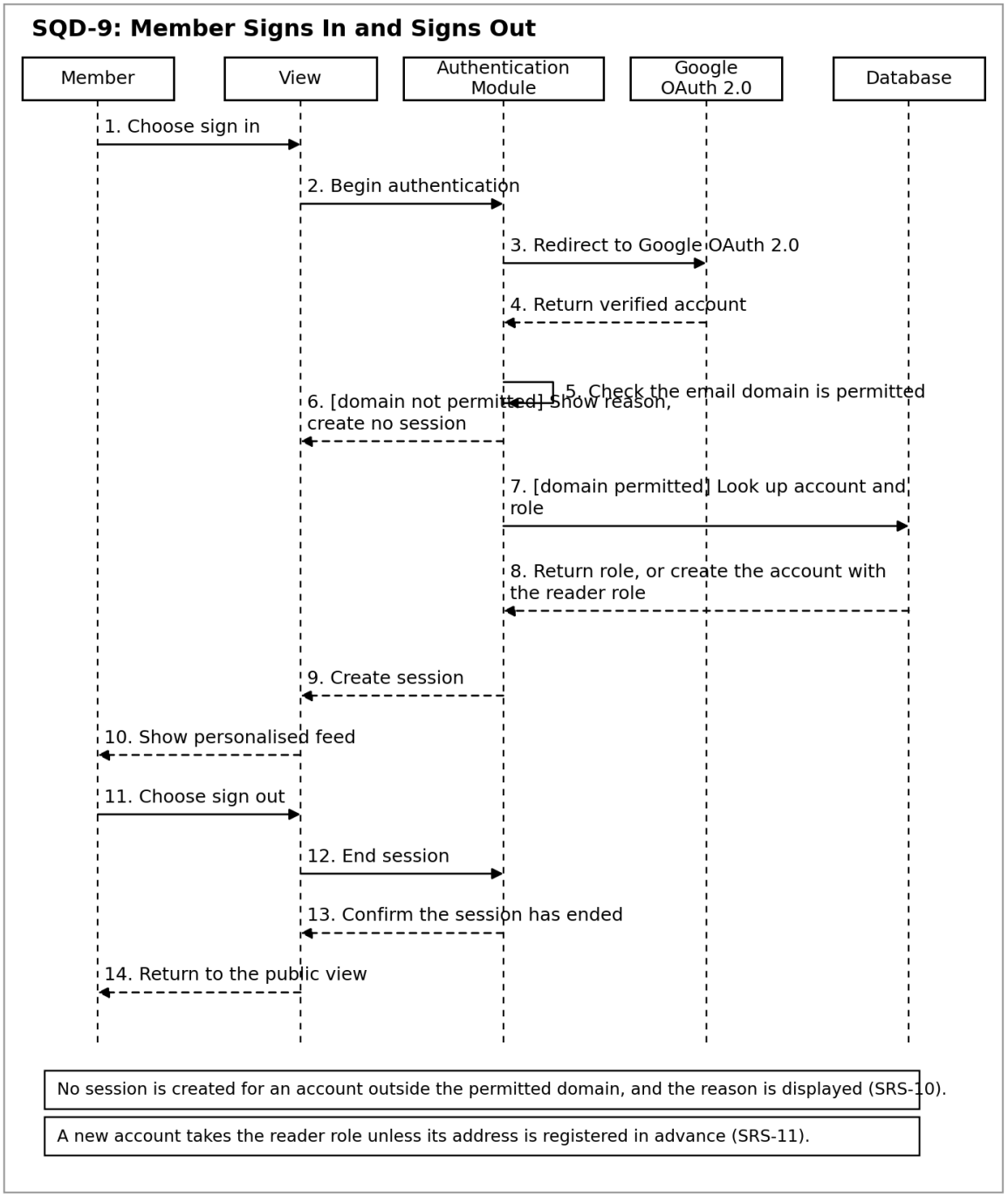
Shows the confirmation the action requires, the authorisation check, retention of the item in the administrator view, and its removal from the student feed, from search, and from the past view.



SQD-9: Member Signs In and Signs Out

Traced requirements: SRS-9, SRS-10, SRS-11, SRS-22

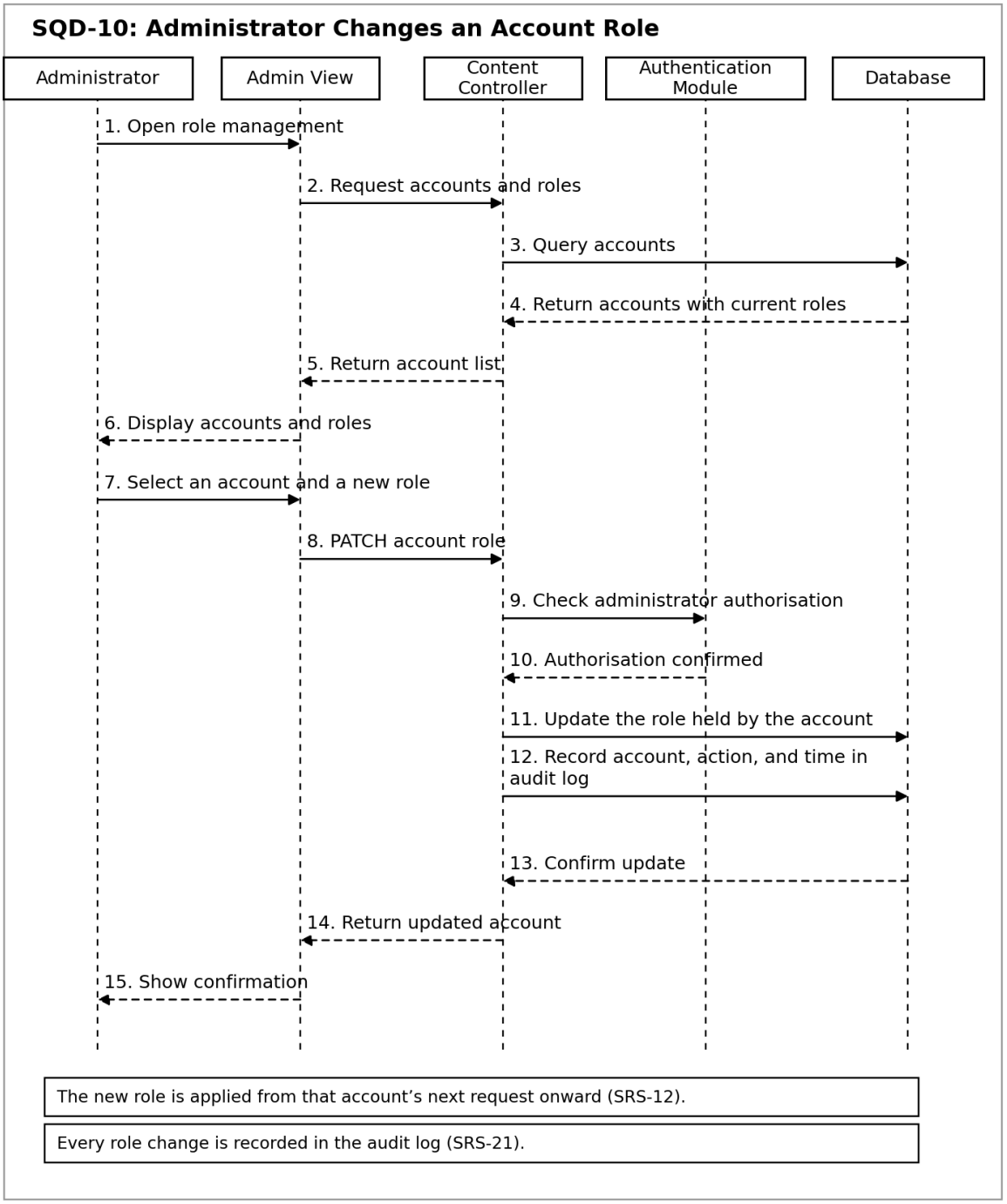
Shows the OAuth 2.0 exchange, the domain check with its refusal branch, the role lookup that follows a successful sign-in, and the path by which a member ends the session and returns to the public view.



SQD-10: Administrator Changes an Account Role

Traced requirements: SRS-12, SRS-21

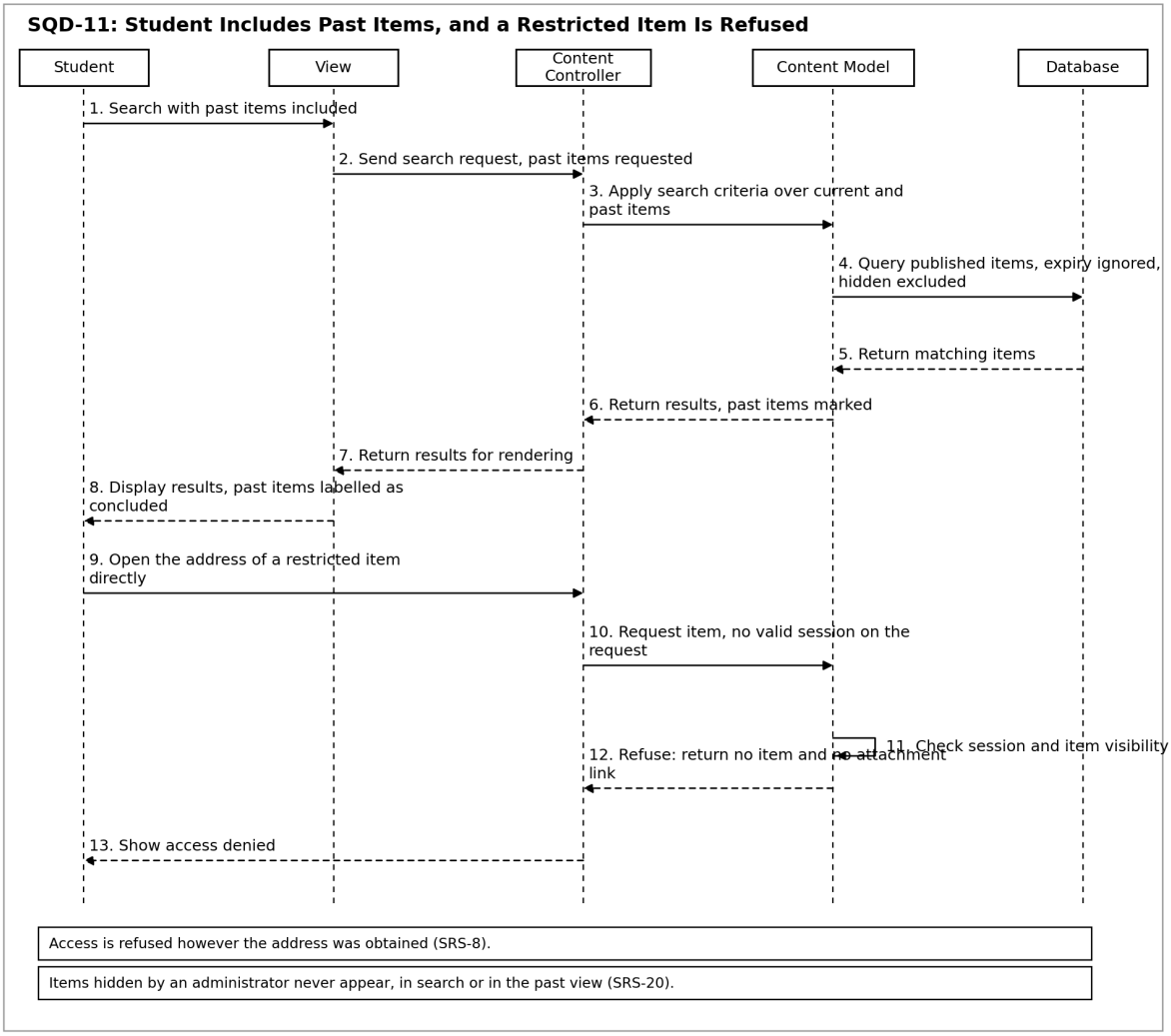
Shows how an administrator changes the role held by an account, how the new role takes effect from that account's next request, and the audit entry written for the change.



SQD-11: Student Includes Past Items, and a Restricted Item Is Refused

Traced requirements: SRS-5, SRS-8

Shows two paths: a search in which the student asks for past items and receives them labelled as concluded, and a request for a restricted item that arrives without a valid session and is refused.



13. Traceability Matrix

Sub-Goal	Use Case	User Story	URS	Activity Diagrams	SRS
SG-1	UC-1, UC-6	US-1, US-9, US-13	URS-1, URS-2, URS-9, URS-14	AD-2	SRS-1, SRS-2, SRS-23, SRS-25
SG-2	UC-5, UC-10	US-5	URS-7	AD-1	SRS-14, SRS-15, SRS-16
SG-3	UC-4, UC-6, UC-9, UC-11	US-4, US-6, US-8, US-10, US-12	URS-5, URS-6, URS-8, URS-12, URS-13	AD-1, AD-3, AD-4	SRS-9, SRS-10, SRS-11, SRS-12, SRS-13, SRS-17, SRS-21, SRS-22
SG-4	UC-7, UC-8	US-7, US-11	URS-10, URS-11	AD-4	SRS-20, SRS-26
SG-4a	UC-7	US-7	URS-10	AD-4	SRS-19, SRS-24
SG-4b	UC-7	US-7	URS-10	AD-4	SRS-18
SG-5	UC-2, UC-3	US-2, US-3	URS-3, URS-4	AD-2	SRS-3, SRS-4, SRS-5, SRS-6, SRS-7, SRS-8

SRS	Sequence Diagram
SRS-1	SQD-1, SQD-7
SRS-2	SQD-1
SRS-3	SQD-2
SRS-4	SQD-2
SRS-5	SQD-11
SRS-6	SQD-3
SRS-7	SQD-3
SRS-8	SQD-11
SRS-9	SQD-9

SRS-10	SQD-9
SRS-11	SQD-9
SRS-12	SQD-10
SRS-13	SQD-4, SQD-5, SQD-6, SQD-8
SRS-14	SQD-4
SRS-15	SQD-5
SRS-16	SQD-4, SQD-6
SRS-17	SQD-4
SRS-18	SQD-7
SRS-19	SQD-6
SRS-20	SQD-8
SRS-21	SQD-4, SQD-8, SQD-10
SRS-22	SQD-9
SRS-23	SQD-6
SRS-24	SQD-6
SRS-25	SQD-1
SRS-26	SQD-6, SQD-7

14. Non-Functional Requirements

The requirements below constrain how the system behaves rather than what it does. Each one is stated so that it can be checked during the demonstration.

ID	Non-Functional Requirement
NFR-1	The current feed shall be displayed within three seconds of the request, for a feed of up to two hundred items, on the department network.
NFR-2	A keyword search over the title and body shall return results within three seconds for a store of up to five thousand content items.
NFR-3	The system shall support at least one hundred signed-in members browsing concurrently without a rise in response time beyond NFR-1.
NFR-4	An attachment of up to twenty megabytes shall be accepted, and the permitted file types shall be restricted to PDF, image, and Office document formats.
NFR-5	The interface shall be usable on a mobile screen of 360 logical pixels in width, since most students reach the hub from a phone.
NFR-6	The system shall run on the current release of Chrome, Firefox, Safari, and Edge.
NFR-7	Session credentials and attachment links shall be transmitted only over HTTPS.
NFR-8	Audit entries shall be retained for at least one academic year and shall not be editable or deletable through the interface.
NFR-9	The hub depends on Google OAuth 2.0 and is unavailable while that service is unavailable; no local password fallback is provided in this iteration.